# MISAKA Network — CLI セットアップガイド

## 📦 配布物

| ファイル | 内容 |
| --- | --- |
| `misaka-net-core-v041-ki-integrated_tar.gz` | L1ブロックチェーンコア (Rust workspace, 17 crates) |
| `misaka-explorer.tar.gz` | ブロックエクスプローラー (Next.js 14) |

---

## 1. misaka-net-core (Rust)

### 動作要件

- Rust 1.75+ (nightly 推奨)
- OpenSSL dev headers
- pkg-config

### セットアップ

```
# 展開
tar xzf misaka-net-core-v041-ki-integrated_tar.gz
cd misaka-net-core

# ビルド
cargo build --release

# テスト実行
cargo test --workspace

# 特定クレートのみテスト
cargo test -p misaka-pqc
cargo test -p misaka-mempool
cargo test -p misaka-consensus
```

## Workspace 構成

```
crates/
  misaka-types/        型定義 (UTXO, Stealth, Block)
  misaka-pqc/          耐量子暗号 (ML-DSA-65, ML-KEM-768, LRS)
  misaka-crypto/       暗号プリミティブ統合
  misaka-execution/    トランザクション実行
  misaka-storage/      UTXO Set ストレージ
  misaka-mempool/      Mempool (KI proof 検証付き)
  misaka-consensus/    BFT コンセンサス + ブロック検証
  misaka-p2p/          P2P ネットワーク
  misaka-tokenomics/   トークン経済
  misaka-governance/   ガバナンス
  misaka-bridge/       クロスチェーンブリッジ
  misaka-mev/          MEV 保護
  misaka-rpc/          JSON-RPC サーバー
  misaka-node/         ノードバイナリ
  misaka-test-vectors/ テストベクタ
```

## v0.4.1 主な変更点

- `pq_ring.rs` : Poly 二重定義を解消、serde_with で [i32; 256] シリアライズ対応
- `ki_proof.rs` : Key Image Correctness Proof (Σ-protocol) 完全実装
  - `prove_key_image()` — Fiat-Shamir 変換 + rejection sampling
  - `verify_key_image_proof()` — コミットメント再構成 + FS ハッシュ検証
  - `KiProof::to_bytes()` / `from_bytes()` — 576B 固定長シリアライズ
- mempool + consensus crate との KI proof 統合完了

## CLI コマンド例

```
# ノード起動 (実装完了時)
cargo run -p misaka-node -- --config node.toml

# テストネットオーケストレーター
cargo run -p misaka-testnet -- --validators 4 --duration 60

# 単体テスト (KI proof)
cargo test -p misaka-pqc -- test_ki_proof_roundtrip --nocapture

# 統合テスト (full transfer pipeline)
cargo test -p misaka-pqc -- test_full_transfer_with_mempool_verification --nocapt
```

## 2. misaka-explorer (Next.js)

### 動作要件

- Node.js 18+
- npm 9+

### セットアップ

```
# 展開
tar xzf misaka-explorer.tar.gz
cd misaka-explorer

# 依存インストール
npm install

# 環境設定
cp .env.example .env.local

# 開発サーバー起動 (Mock モード)
npm run dev
```

→ http://localhost:3000 でエクスプローラーが開く

### 環境変数

`.env.local` を編集:

```
# Mock データで動作 (デフォルト)
NEXT_PUBLIC_USE_MOCK=true

# 実 RPC に接続する場合
NEXT_PUBLIC_USE_MOCK=false
NEXT_PUBLIC_MISAKA_RPC_URL=http://your-node:3001
```

### ページ一覧

| パス | 内容 |
|---|---|
| `/` | ダッシュボード (Stats, Chart, Recent Blocks/Txs) |

| | |
|---|---|
| `/blocks` | ブロック一覧 (ページネーション) |
| `/blocks/:id` | ブロック詳細 (height or hash) |
| `/txs` | トランザクション一覧 |
| `/txs/:hash` | トランザクション詳細 |
| `/address/:addr` | アドレスページ (プライバシー保護表示) |
| `/validators` | バリデーター一覧 |
| `/validators/:id` | バリデータ詳細 |

### 機能

- 自動リフレッシュ (15秒トグル)
- ハッシュコピーボタン
- Raw JSON ビューア (Block/Tx 詳細)
- スケルトンローディング (全ページ)
- エラーバウンダリ + リトライ (全ルート)
- プライバシー保護メタデータのみ表示
- レスポンシブダークUI

### プロダクションビルド

```
npm run build
npm start
```

---

## 3. 技術スタック概要

### ブロックチェーンコア

| 技術 | 用途 |
|---|---|
| Falcon-512 / ML-DSA-65 | 署名 (FIPS 204) |
| | |

| | |
|---|---|
| ML-KEM-768 / Kyber-768 | 鍵カプセル化 (FIPS 203) |
| LaRRS (LRS-v1) | 格子環署名 (R_q = Z_12289[X]/(X^256+1)) |
| NTT | O(n log n) 多項式乗算 |
| SHA3-256/512 + HKDF | ハッシュ + 鍵導出 |
| XChaCha20-Poly1305 | ステルスアドレス暗号化 |

### エクスプローラー

| 技術 | 用途 |
|---|---|
| Next.js 14 (App Router) | フレームワーク |
| TypeScript | 型安全 |
| Tailwind CSS | スタイリング |
| Recharts | チャート |
| DM Sans + JetBrains Mono + Outfit | フォント |

---

## 4. トラブルシューティング

### Rust ビルドエラー

```
# OpenSSL が見つからない場合
sudo apt install libssl-dev pkg-config

# pqcrypto 関連
sudo apt install cmake clang
```

### Explorer npm install エラー

```
# Node バージョン確認
node --version  # 18+ 必須

# キャッシュクリア
npm cache clean --force
rm -rf node_modules package-lock.json
```

```
npm install
```

---

## 5. プライバシーポリシー

MISAKA はプライバシー重視のブロックチェーンです:

- リング署名で送信者の匿名性を保護
- ステルスアドレスで受信者を秘匿
- エクスプローラーは公開検証可能なメタデータのみ表示
- リングメンバーやステルス受信者の特定は行わない

---

**MISAKA Network** — Post-Quantum Native Privacy Blockchain